



Node

Web Programming
Lecture 15
Mahhek Khan

File Structure

✓ MERN-BACKEND

✓ config

JS db.js

✓ controllers

JS userController.js

✓ models

JS User.js

> node_modules

✓ routes

JS userRoutes.js

⚙ .env

JS index.js

{ } package-lock.json

{ } package.json



MongoDB connection (config/db.js)

```
// import mongoose library
const mongoose = require('mongoose');

// function to connect to MongoDB
const connectDB = async () => {
  try {
    // connect to database using connection string from .env
    await mongoose.connect(process.env.MONGO_URI);

    // message if connection is successful
    console.log('MongoDB connected');
  } catch (err) {
    // print error if connection fails
    console.error(err.message);

    // stop the server if DB connection fails
    process.exit(1);
  }
};

// export the function so it can be used in other files
module.exports = connectDB;
```

MONGO_URI=mongodb://localhost:27017/myapp



In index.js now

```
// import express
const express = require("express");

// import database connection
const connectDB = require("../config/db");

const app = express();

// connect to database
connectDB();

// middleware
app.use(express.json());

// test route
app.get("/", (req, res) => {
  res.send("Server running");
});

// start server
const PORT = 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```



Flow

Server starts



index.js runs



connectDB() is called



config/db.js connects to MongoDB



Server ready to accept requests

`config/db.js` contains the MongoDB connection logic, and `index.js` imports and calls it when the server starts.



Middleware

- `app.use()` is used to run middleware functions.
- Middleware is a function that runs when a request comes to the server, before the request reaches the route handler.
- Flow of a request:
 - Client request → Middleware → Route handler (controller) → Response
- Middleware can:
 - modify the request
 - check authentication
 - parse data
 - log information



Example – express.json() Middleware

```
app.use(express.json())
```

`express.json()` is a middleware that reads JSON data sent in the request body.

Example request body:

```
{  
  "name": "Apple",  
  "quantity": 10  
}
```

The middleware converts this JSON into a JavaScript object and stores it in:

```
req.body
```

So inside the route/controller we can access:

```
req.body.name
```

```
req.body.quantity
```

models/item.js

```
// import mongoose
const mongoose = require("mongoose");

// define schema (structure of item document)
const itemSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  quantity: {
    type: Number,
    required: true
  }
});

// create model from schema
const Item = mongoose.model("Item", itemSchema);

// export the model
module.exports = Item;
```


itemController.js

```
const Item = require("../models/item");

exports.getItems = async (req, res) => {
  try {
    const items = await Item.find();
    res.json(items);
  } catch (err) {
    res.status(500).send(err.message);
  }
};

exports.updateItem = async (req, res) => {
  try {
    const item = await Item.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    );

    res.json(item);
  } catch (err) {
    res.status(500).send(err.message);
  }
};

exports.deleteItem = async (req, res) => {
  try {
    await Item.findByIdAndDelete(req.params.id);
    res.send("Item deleted successfully");
  } catch (err) {
    res.status(500).send(err.message);
  }
};
```



What **req** and **res** mean

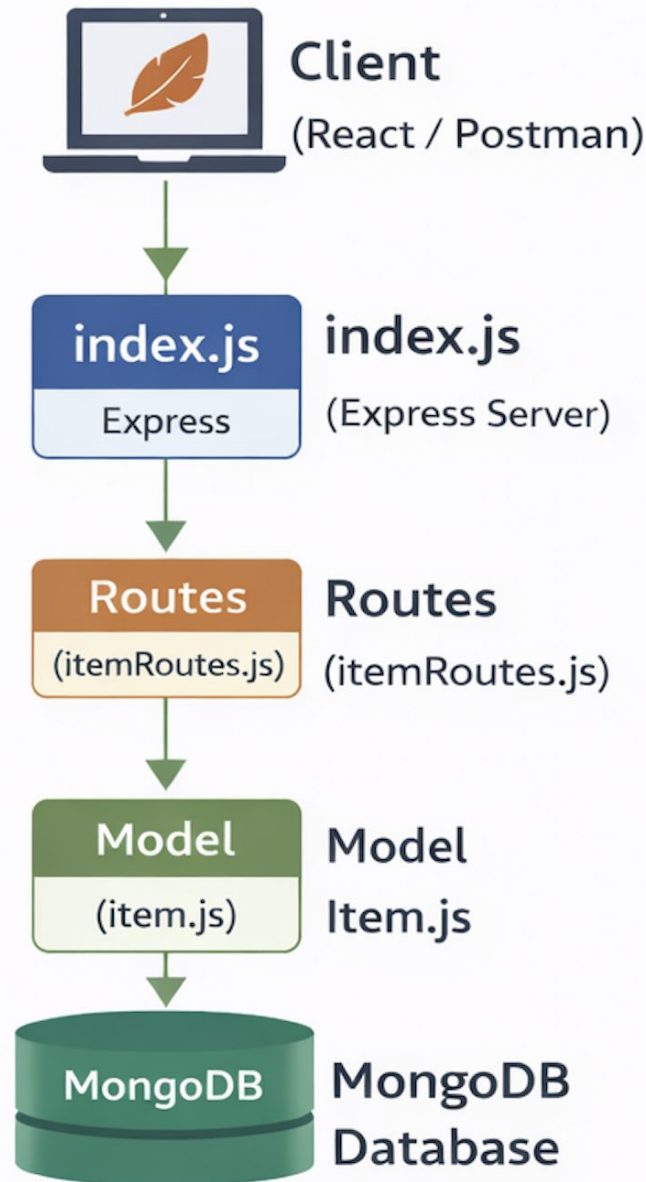
What **req** and **res** mean

(req, res)

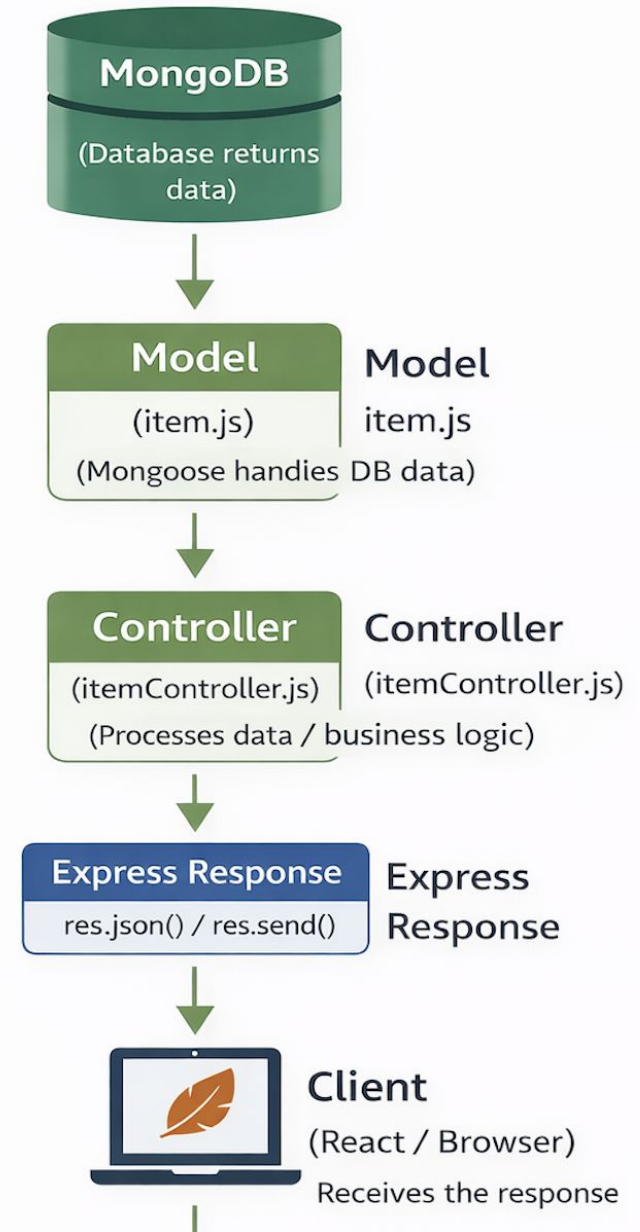
- **req** → request from the client
- **res** → response sent back to the client

- req.params.id
- req.body
- res.json()
- res.send()

REQUEST FLOW



RESPONSE FLOW



router.put("/:id", updateItem)



PUT /api/items/123

req.params.id

123



req.body

This contains **data sent by the client**.

Example request body:

```
{  
  "name": "Laptop",  
  "quantity": 5  
}
```

Accessible as:

req.body

req.body.name

req.body.quantity

becauseeeee.. we used middleware:

```
app.use(express.json());
```



Express Response Methods

In Express, the server sends responses to the client using the `res` object.

Common response methods:

- ▶ `res.send()`
- `res.json()`
- `res.status()`

These methods are used inside controllers to send data back to the client.



res.send()

res.send() sends a response of any type.

Example:

```
res.send("<p>Server running</p>")
```

It can send:

- text
- HTML
- JSON
- numbers

Example:

```
res.send("Hello")
```

```
res.send({ name: "Ali" })
```



res.json()

res.json() sends data as JSON.

Example:

```
res.json({ name: "Laptop", price: 500 })
```

The response automatically becomes JSON.

Used most often in APIs.



res.status()

res.status() sets the HTTP status code.

Example:

```
res.status(200).json(data)
```

Common status codes:

200 → success

201 → created

400 → bad request

404 → not found

500 → server error



Valid combos

```
res.send("Hello")
```

```
res.json({ name: "Ali" })
```

```
res.status(200).json(data)
```

```
res.status(404).send("Item not found")
```

Pass Parameter as body to api



#1: Client-side (React, etc.): You send a request like: (Create)

POST /items

Content-Type: application/json

```
{  
  "name": "Notebook",  
  "quantity": 10  
}
```

#2: Server-side (Node.js with Express): The server receives the request.

```
const { name, quantity } = req.body;
```

This line extracts **name** and **quantity** from the request body.

#3: Passing to Model

Then, Mongoose uses these values to create a new document:

```
const newItem = new Item({ name, quantity });  
await newItem.save();
```

Pass Parameter as body to api



#1: Client-side (React, etc.): You send a request like: (Update)

```
PATCH /items/12345678
Content-Type: application/json
{
  "name": "Updated Item",
  "quantity": 20
}
```

#2: Server-side (Node.js with Express): The server receives the request.

```
const { name, quantity } = req.body;
```

#3: Passing to Model

```
const item = await Item.findByIdAndUpdate(
  req.params.id, // The item ID passed in the URL
  { name, quantity }, // The data passed in the body of the request
  { new: true, runValidators: true }
);
```

Return the **updated document** instead of the old one.

if parameters doesn't match

```
{  
  "productName": "Apple",  
  "qty": 5  
}
```

```
exports.updateItem = async (req, res) => {  
  try {  
    // ✅ Mapping frontend → DB fields  
    const updateData = {  
      name: req.body.productName,  
      quantity: req.body.qty  
    };  
  
    const item = await Item.findByIdAndUpdate(  
      req.params.id,  
      updateData,  
      { new: true, runValidators: true }  
    );  
  
    if (!item) {  
      return res.status(404).json({  
        success: false,  
        message: 'Item not found'  
      });  
    }  
  
    res.status(200).json({  
      success: true,  
      data: item  
    });  
  
  } catch (error) {  
    res.status(500).json({  
      success: false,  
      message: error.message  
    });  
  }  
};
```



query string

<http://localhost:5000/api/items?category=electronics&page=2>

```
exports.getItems = (req, res) => {  
  const category = req.query.category;  
  const page = req.query.page;  
  
  res.send(`Category: ${category}, Page: ${page}`);  
};
```

So we studied

- ▶ req.params
- ▶ req.query
- ▶ req.body